Prefetching and Tail Latency in Lock-Free Inter-Core Queues: Effects of Access Pattern, Placement, and Offered Load

Abstract

Inter-core queues form an event-transfer boundary in latency-sensitive pipelines, but queue and prefetch studies usually optimize different response variables. Concurrent-queue work has measured throughput, operation latency, and, in recent cases, upper-tail item-transfer latency, whereas linked-data prefetch research has primarily measured program runtime or memory stalls. The reviewed literature does not jointly isolate queue-design policy, verified hardware- and software-prefetch policies, producer–consumer topology, and open-loop upper-tail latency. This paper develops a pre-specified experiment comparing a bounded single-producer/single-consumer ring with a fixed-arena linked one-producer/one-consumer FIFO, including its node recycler, under fixed producer-home data placement, immutable matched event-record access, common logical capacity, matched working-set residency class, and sub-saturation offered load. The ring uses arithmetic slot lookahead; the linked policy uses a one-hop read hint on a successor drawn from a pre-generated seed-permuted node cycle. The primary estimand concerns each complete representation and reuse policy; address-pattern diagnostics support, but cannot alone establish, a pointer-dependence explanation, and package footprints need not be equal. The design uses independently verified hardware-prefetch states, pre-generated open-loop arrivals, a deterministic warm-start reset, ordered invocation/linearization/response observations joined by accepted ordinal, a non-interpolated run-level $p_{99.9}$ response, separate H1/H2 complete-block max-T families, and sealed validation sized over a complete pre-selection family. Queue-full returns and genuine effective-tail insufficiency are retained as observed outcomes that separately gate the confirmatory estimand; a treatment-blind matrix-level feasibility bound precedes strict-zero-loss confirmation. The intended contribution is an estimable operational model and a reproducible pre-freeze methodology; no empirical outcome is asserted, and confirmatory execution remains blocked until the practical bound and other mandatory protocol items are frozen.

1 Introduction

Latency-sensitive event pipelines often hand an event from a producer on one core to a consumer on another through a concurrent FIFO. The queue is therefore both a synchronization object and a memory-access path. Its contribution to delay can change when an event or queue line is supplied by a private cache, a shared last-level cache (LLC), another coherence domain, or remote memory. These effects need not appear in the median alone. Work on server tail latency has shown that operating-system activity, power state, thread placement, and non-uniform memory access can widen upper quantiles even when ordinary operation is fast [10]. This paper does not claim hard real-time bounds; it concerns observed latency distributions in low-latency event pipelines.

Queue representation changes what a prefetcher can predict. A bounded ring exposes an arithmetic stream of slot addresses, and cache-conscious SPSC designs use ownership separation and batching to reduce coherence traffic [6, 24, 19, 25]. A linked FIFO discovers a successor through a load from the current node, but a contiguous arena and FIFO reuse can accidentally turn its realized addresses into a stride. The protocol therefore supplies cache-line-aligned nodes in a fixed

seed-permuted cycle and accepts an address-dependence interpretation only after the measured successor-delta trace passes its pattern gate. Compiler, hardware, and jump-pointer studies identify serial address dependence as an obstacle to timely linked-data prefetching [11, 20, 21, 26]; an arbitrary address several nodes ahead is not available in the unchanged linkage. Consequently, a numeric "prefetch distance" is not a representation-independent treatment.

The queue literature has also moved beyond mean throughput. Relaxed Concurrent Queues (RCQs) report the 99.9th percentile of operation and item-transfer latency in closed and rate-imbalanced settings [8]; modern bounded queues such as SCQ and wCQ establish strong memory-footprint and progress properties under MPMC use [17, 18]; and NBLFQ targets the low-contention region of a bounded MPMC queue [3]. These studies make a throughput-only account insufficient, but they do not control hardware and software prefetch as experimental factors under pre-generated open-loop arrivals. Conversely, prefetch studies expose timeliness, accuracy, pollution, bandwidth, and hardware–software interaction, but do not measure end-to-end FIFO tail latency [23, 4, 9, 12].

The resulting gap is narrow. Prior work has studied concurrent-queue latency and linked-data prefetching separately, but the reviewed literature does not jointly isolate queue representation, independently verified hardware- and software-prefetch policies, producer–consumer topology, and open-loop upper-tail latency. This statement is limited to the sources and search process documented with this draft; it is not a claim that no related experiment exists.

The confirmatory study asks three questions:

RQ1 How do verified hardware-prefetch configurations and feasible, family-specific software-prefetch policies modify $p_{99.9}$ end-to-end latency across a bounded ring package and a seed-permuted linked-FIFO-plus-recycler package at common logical capacity and matched working-set residency class?

RQ2 How do producer–consumer placement, working-set residency, and offered load change the direction and magnitude of a prefetch effect and the useful ring lookahead?

RQ3 Can independently precision-gated training and held-out measurements support a platform-conditioned selection rule for a predeclared operational subset?

The corresponding hypotheses are predeclared in Section 4.4. H1 tests differences between within-family software effects, queue-specific hardware effects, and hardware–software interactions; it never treats ring lookahead and linked one-hop as a shared numeric level. H2 tests the same defined effects across placement, working-set-residency, and load contrasts. H3 is conditionally confirmatory: its deterministic training rule and held-out block count have an independent prospective precision gate, and failure to meet that gate leaves RQ3 unresolved. The practical bound δ_* is not frozen because the repository contains no application latency budget or engineering authorization. Accordingly, this manuscript is a pre-freeze protocol draft; confirmatory execution, final replication counts, and H3 validation cannot begin until that decision is recorded.

This pre-experiment paper contributes four items. First, it states invocation, linearization, response, and admission semantics that preserve open-loop offered load when a producer is late or a bounded queue is full; immutable event records give both packages the same safe read-only address stream, and ordered private observations are joined after the run by accepted ordinal. Full/drop counts and genuine effective-tail insufficiency remain observed outcomes, while separate gates control whether a required contrast is estimable. Second, it separates bundled ring slot lookahead from one-hop linked-node prefetch and controls the linked node cycle, recycler, producer-home memory policy, and deterministic logical reset after warm-up. Third, it gives a full-rank run-level response model and an exact registry of seven H1 and twenty H2 contrasts in a split-plot design containing 180 cells per complete block. The two families receive separate prospective max-T sizing and R_{12} is their maximum; the earlier 3,600–5,400-run range is only a planning envelope conditional on freezing δ_* and satisfying both rules. Fourth, the protocol defines complete-block replacement rather

than cell replacement and a sealed access chronology whose H3 sizing is independent of selection: training covers all 270 unordered pair–context differences and validation the complete 540-member ordered pre-selection family. Role-assigned blocks become available to H1/H2 only after selection is frozen, validation is unsealed and evaluated, and the H3 access record is sealed. MPSC contention, mutable event preparation and ownership handoff, batching, bursty arrivals, background interference, alternative page placement, memory-order sensitivity, same-core simultaneous multithreading (SMT), and overload remain staged extensions.

2 Background

2.1 Queue semantics and experimental objects

A FIFO implementation is *linearizable* when each completed operation can be assigned a point between invocation and response such that the resulting order satisfies the sequential object. Progress is separate: lock-freedom guarantees system-wide progress, whereas wait-freedom guarantees completion for each active operation under the algorithm's assumptions. An empty dequeue or full bounded enqueue may still return failure; the progress property concerns the termination of that try-operation, not guaranteed admission.

The confirmatory objects are both one-producer/one-consumer FIFOs. The ring is the cache-line-separated bounded SPSC circular buffer described by Torquati, derived from Lamport's SPSC construction [24]. Producer and consumer own separate indices, and a distinguished empty slot value makes full and empty tests local to individual slots. The linked object uses Torquati's dSPSC node-linking and SPSC node-cache discipline, but instantiates its allocation domain as a fixed, cache-line-aligned arena [24]. Before warm-up, a seeded permutation chooses the sentinel and the logical FIFO order in which the remaining nodes enter the ring-based SPSC recycler; successful transfers then repeat that permutation without randomization or sorting on the measured path. Exhausting the arena makes `try-enqueue` return full rather than invoke a general allocator. Because this bounded and ordered allocation instance is an experimental adaptation, its linearizability, FIFO order, progress, node-cycle, and reuse properties must be re-established by the validation gates in Section 6.9.

The comparison is deliberately not a contest among all queue algorithms. Michael and Scott provide the conventional MPMC linked reference design [14], while hazard pointers illustrate why safe reuse is an independent concern for general lock-free linked structures [13]. SCQ, wCQ, RCQ, and NBLFQ address different combinations of strictness, bounded memory, progress, and concurrency [17, 18, 8, 3]. They motivate later contention work but would introduce cardinality and algorithmic factors into the mandatory representation comparison.

2.2 Coherence, placement, and access pattern

Private caches, an LLC, coherence directories or snoop domains, and NUMA memory form distinct access paths. A producer publishing a slot or node and a consumer reading it can transfer cache-line ownership even when the payload itself is resident. False sharing adds transfers when independently owned control fields share a line. The latency of synchronization primitives and cache-to-cache movement depends on topology and architecture [15, 2]; NUMA-aware data-structure work likewise shows that placement is part of the object presented to hardware, not merely an environmental detail [1].

Ring traversal supplies regularly increasing slot addresses, so stream prefetchers may learn it and software can calculate a future slot without first reading intervening slots. Linked traversal

exposes the next address only after the current node has been read, but linked syntax alone does not guarantee irregular physical addresses. Stage A therefore records successor-address deltas and rejects a node-order seed that creates excessive unit or repeated stride; only a passing seed-permuted cycle is described as dependent non-sequential traversal. A one-hop prefetch can use the newly obtained successor, but a two- or three-hop address requires dependent traversal. Jump pointers can break the dependence at the cost of extra fields and maintenance [21]; such a structure is a different queue design, not a neutral prefetch level for the original list.

Hardware and software prefetch can help, interfere, or consume shared resources. Timeliness asks whether a requested line arrives before demand; coverage asks how many demand misses are anticipated; accuracy asks how many fetched lines are subsequently used. Pollution and bandwidth cost capture displacement and traffic imposed on other accesses. Feedback and coordinated-prefetch studies show why these mechanisms must be interpreted together rather than as independent switches [23, 4, 9]. Software prefetch instructions are hints, and their observable behavior varies by processor and memory distance [12]. A requested hardware-prefetch state is therefore a treatment only after readback or an independent behavioral check establishes that it differs from the platform default.

2.3 Ordering and latency observation

Cache coherence controls how copies of a cache line are reconciled; a language memory model controls which inter-thread observations are permitted. One does not substitute for the other. Stage A event records are initialized once before warm-up and remain immutable, so enqueue publication orders only the selected record pointer and queue metadata; it does not publish a newly prepared payload. Release publication and acquire observation protect the slot or link handoff, and linked-node reclamation cannot precede the consumer's last node access. The consumer reads the immutable index and aligned 64-bit payload and updates a private rolling checksum before its completion timestamp; generated-code and post-run checksum checks make this fixed action observable without adding volatile or atomic record fields. Mutable payload preparation and record ownership transfer are separate Stage C treatments. Alternative queue orderings are likewise Stage C sensitivity treatments. Weak-memory producer–consumer analysis and the queue sources motivate this mapping, but the future implementation must establish it under the selected language model before performance measurements are accepted [24].

End-to-end latency and queue operation service time answer different questions. End-to-end latency starts at a logical scheduled arrival. Its additive partition ends admission and queue residence at queue-specific enqueue and dequeue linearization boundaries, then ends delivery after the consumer action. Producer lateness, pointer lookup, enqueue service, dequeue service, and the checksum action are nested diagnostics; adding them to that partition would double count time. Retaining invocation, linearization, and response boundaries also avoids defining residence from cross-core operation responses that may overlap. Open arrivals must be independent of prior completion [22]. If a generator waits for an operation to finish before scheduling the next, stalls suppress precisely the arrivals that should expose the tail, producing coordinated omission [5]. The timestamp model in Section 4 preserves that distinction.

3 Related Work

3.1 Inter-core and concurrent queues

Cache-conscious SPSC queues reduce communication overhead through index separation, temporal displacement, local caching, or cache-line batches [6, 24, 19, 25]. Their evaluations establish that layout and placement affect core-to-core transfer cost, but they predominantly report throughput or average operation latency. BatchQueue explicitly trades increased item latency for cache-line batching [19], illustrating why batching is not part of the single-item confirmatory core.

MPMC work emphasizes scalability, semantics, progress, and memory use. Michael–Scott is a linked baseline [14]; LCRQ and SCQ use ring segments or bounded rings to improve behavior on modern processors [16, 17]; wCQ adds wait-freedom with bounded memory [18]. NBLFQ is a bounded MPMC ring designed for low contention and evaluates throughput and median communication latency across four architectures [3]. RCQ is the closest latency-oriented study located: it reports 99.9th-percentile operation, wait, and task latency under closed and rate-imbalanced workloads [8]. It does not, however, make verified hardware-prefetch state or family-specific software prefetch a factor, and its "open system" changes thread processing delay rather than preserving a pre-generated schedule of logical arrivals.

3.2 Prefetching linked and regular accesses

Software prefetching for recursive structures identifies the lack of an independently available future address as a limiting dependence [11]. Dependence-based hardware traversal and jump-pointer designs obtain more lead time through speculative traversal or added metadata [20, 21]. Augur uses semantic information in a hardware temporal prefetcher for linked-data workloads [26]. These mechanisms support the premise that linked access differs from a stride, but their response is program speed or memory-stall behavior rather than queue residence or end-to-end tail latency.

Studies of adaptive and coordinated prefetch control show that coverage alone is inadequate when inaccurate or aggressive requests consume cache and bandwidth resources [23, 4]. Lee, Kim, and Vuduc directly analyze cooperation and interference between hardware and software policies [9]. Mahling, Weisgut, and Rabl further demonstrate that a software hint can have processor-dependent behavior under long-latency memory [12]. The present protocol therefore treats hardware state as a verified whole-plot factor and reports mechanism counters as diagnostics, not causal substitutes for randomized latency contrasts.

3.3 Tail-latency methodology

Li et al. measure median and 99.9th-percentile service latency under open-loop Poisson clients and identify placement, background work, and power management as sources of tail variation [10]. Schroeder et al. distinguish open from closed generators [22], while Friedrich et al. show how synchronous scheduling omits arrivals during a long response [5]. These studies motivate the logical-arrival clock and count reconciliation in this design. Kalibera and Jones motivate estimating uncertainty at the level where independent variation enters and choosing repetition counts from precision rather than convention [7].

Tables 1 and 2 compare individual papers. A check mark records only an explicitly evaluated dimension. Each paired entry in the PF column reports hardware/software control; Top./OL reports topology or NUMA/open-loop coverage. "Open-loop" means that arrivals are scheduled independently of operation completion and is not assigned merely because a paper calls a load-imbalanced experiment open.

Table 1: Concurrent-queue studies. A dash denotes a dimension not explicitly evaluated. PF is reported as HW/SW and Top./OL as topology/open-loop.

Work	Object	Main response	Tail	PF	Top./OL	Limitation relative to this study
Giacomoni et al. [6]	bounded SPSC ring	operation latency, pipeline speed	–	–/–	✓/–	no tail quantiles or prefetch treatments
Torquati [24]	ring, linked, and block SPSC	average operation latency	–	–/–	✓/–	no upper-tail or controlled offered load
Preud'homme et al. [19]	batched SPSC	transfer time, throughput	–	–/–	–/–	batching changes latency semantics
Wang et al. [25]	bounded SPSC ring	throughput, stability	–	–/–	✓/–	no tail or prefetch factor
Michael and Scott [14]	linked MPMC FIFO	throughput	–	–/–	–/–	reclamation and tail latency outside scope
Morrison and Afek [16]	ring-segment MPMC FIFO	throughput	–	–/–	–/–	no prefetch or end-to-end response
Nikolaev [17]	bounded MPMC ring	throughput, memory use	–	–/–	–/–	no tail quantiles or topology factor
Nikolaev and Ravindran [18]	bounded wait-free MPMC ring	throughput, memory use	–	–/–	–/–	no prefetch or end-to-end tail
Kappes and Anastasiadis [8]	strict and relaxed array queues	operation/item latency, throughput	✓	–/–	–/–	rate imbalance rather than pre-generated arrivals; no PF control
Denis and Goedefroit [3]	bounded MPMC ring	throughput, median network latency	–	–/–	–/–	low-contention focus; no tail or PF control

Table 2: Prefetch and tail-latency studies, using the notation of Table 1.

Work	Object	Main response	Tail	PF	Top./OL	Limitation relative to this study
Luk and Mowry [11]	lists, trees, graphs	runtime, memory stalls	–	–/✓	–/–	not a concurrent queue
Roth and Sohi [21]	linked structures with jump pointers	runtime, memory stalls	–	✓/✓	–/–	extra metadata changes the data structure
Lee et al. [9]	application memory accesses	runtime, cache behavior	–	✓/✓	–/–	no queue or topology-controlled tail
Xue et al. [26]	linked-data workloads	runtime, prefetch accuracy	–	✓/–	–/–	hardware proposal; no FIFO latency
Li et al. [10]	network services	response latency	✓	–/–	✓/✓	no queue-representation or PF treatment

The proposed study occupies the conjunction of the table's sparse columns, not an unqualified claim of novelty. It narrows the object comparison to two 1P1C FIFOs so that representation, prefetch feasibility, placement, working set, and open-loop $p_{99.9}$ can be crossed without mixing in

producer contention. Stage B can then test whether an MPMC algorithm instantiated as MPSC changes those conclusions.

4 Problem Formulation

4.1 System, arrivals, and controlled address streams

One producer and one consumer are pinned to distinct physical cores. The producer transfers pointers to fixed-size event records through queue package $q \in \{\text{ring}, \text{linked}\}$. Both packages use the same frozen logical capacity C: the ring has C addressable slots, while the linked package has a $C + 1$-node arena including its sentinel and a bounded ring-based SPSC recycler with operational capacity C. Their allocated byte footprints need not be equal. Queue occupancy $O(t)$ is the number of accepted, not-yet-consumed events at time t and is observed rather than set as a Stage A factor.

Before a run, a seeded generator creates the complete ordered arrival schedule $\{a_i\}$ independently of queue completions. Stage A uses exponential interarrival times at fixed rate λ, giving a stationary open-loop Poisson schedule. Its integer-tick deadlines occupy the half-open horizon $[t_0, t_0 + H)$; the schedule records an exact rational nominal rate, count, namespace derivation, encoding, overflow rule, and checksum. A *logical arrival* exists at a_i even when the producer is busy. The producer never shifts a future a_j in response to lateness; when several deadlines have passed, it attempts due events in sequence and records producer lateness. The primary admission policy is one try-enqueue per logical arrival. Full is an observed drop outcome without retry or backpressure: it is retained, counted, and never by itself invalidates or triggers replacement of a run. Stage A nevertheless targets a strict zero-loss sub-saturation regime. Calibration and pilot data are used, before matrix freeze, to choose C and offered loads for which a prospective simultaneous bound supports completing the whole planned matrix without a full outcome. If $N_{\text{full}} > 0$ later occurs in a required confirmatory cell, the run and its accepted-event latency summary remain reported, but the cell is outside the predeclared regime and any confirmatory contrast requiring it is blocked rather than recomputed after deletion.

The linked arena is one page-aligned, virtually contiguous allocation backed on the producer's NUMA node; each node occupies an integral number of cache lines and begins on a cache-line boundary. Physical contiguity is not assumed. Before first use, a separate address-order seed generates a permutation π of all $C + 1$ node indices. Node π_0 is the initial sentinel and the recycler is seeded in FIFO order with $\pi_1, \ldots, \pi_C$. Because each consumed old sentinel is appended to that recycler, zero-loss execution realizes the same cyclic permutation without random-number generation, sorting, allocation, or permutation on the measured path. The arena mapping and π persist across paired linked off/on and hardware-state runs within a temporal block.

The event-record pool is a distinct, common arena of C one-cache-line records for all queue packages at a common logical capacity and matched working-set residency class. Every record is initialized before warm-up with an immutable index, one fixed aligned 64-bit payload word, and inert padding; no timestamp, logical sequence number, payload, or ownership field is rewritten during Stage A. A second seed defines the cyclic record-index order. For each logical arrival, the producer performs deterministic index lookup and enqueues the corresponding pointer. After dequeue, the consumer reads the index and payload, combines both into a consumer-private 64-bit rolling checksum using one frozen deterministic fixed-instruction integer-mixing operation, and only then records f_i. The operation has no payload-dependent branch or allocation, is identical across packages, and is verified by generated-code inspection and a post-run checksum; no record field is made volatile, atomic, or mutable merely to preserve the loads. Concurrent references to the same record are safe because the record is immutable. Mutable event preparation, recycling, and

ownership handoff are Stage C extensions. The same mapped pool, record order, and action are used across queue families and paired treatments; queue metadata remains family-specific. The manifest stores both seeds, ordered-index checksums, virtual address-delta checksums, page counts, the frozen mixing-operation identity, and the first-touch and data-home records.

The linked sequence is called *seed-permuted dependent-address traversal* only after a pre-run trace confirms $C + 1$ distinct node lines per cycle, no period shorter than $C + 1$, and no more than 1% unit-stride transitions or 1% occurrences of any single nonzero signed delta. The trace reports the complete successor-delta distribution, unit- and modal-stride fractions, distinct lines and pages, detected period, and equality of paired delta-vector checksums. Physical page-frame order is recorded where the operating system exposes it. Failure causes selection of the next treatment-blind seed before the matrix is frozen; it never causes post-outcome reseeding. If no feasible seed passes, the linked package remains a complete-design comparison but no interpretation is attributed specifically to irregular pointer dependence.

For every run,

$$N_{\text{offered}} = N_{\text{attempted}}, \tag{1}$$

$$N_{\text{attempted}} = N_{\text{accepted}} + N_{\text{full}}, \tag{2}$$

$$N_{\text{accepted}} = N_{\text{consumed}} + O(t_{\text{stop}}). \tag{3}$$

After the measurement window, a drain phase must make $O = 0$. Every logical producer row stores run identity, logical sequence, record index, scheduled arrival, handling start, record-lookup completion, enqueue invocation, completion of the single attempt, and accepted/full outcome; an accepted row additionally stores its enqueue linearization and accepted ordinal. A full row stores no accepted ordinal or enqueue linearization. Every successful consumer row stores run identity, consumed ordinal, observed record index, dequeue invocation, dequeue linearization, dequeue response, and consumer-action completion. Post-run processing constructs the producer-accepted sequence and requires consumer observation k to match accepted event k. The join key is run identity plus accepted ordinal; the repeating record index is a validation field rather than a globally unique event identifier. A count mismatch, unexpected pointer, index mismatch, duplication, omission, forbidden reordering, timestamp-order violation, or derived-equation violation is a correctness failure. A failed join audit produces no joined latency data. End-to-end latency is computed only after reconciliation passes. Overload and retry or backpressure policies belong to Stage C.

4.2 Latency variables

For logical arrival i, let b_i be producer handling start, c_i record-lookup completion, u_i enqueue invocation, and e_i completion of the single try-enqueue response. An accepted row additionally records its queue-specific publication/linearization boundary p_i. For a successful dequeue, let v_i be invocation, d_i removal/linearization, r_i response completion, and f_i completion of the index/payload loads and private rolling-checksum update. Stage A admission includes logical-arrival handling, deterministic record-index lookup, and enqueue publication, but no mutable payload preparation. The exact derived quantities are

$$L_{\text{late},i} = b_i - a_i, \qquad S_{\text{lookup},i} = c_i - b_i, \tag{4}$$

$$S_{e,i} = e_i - u_i, \qquad D_{\text{adm},i} = p_i - a_i, \tag{5}$$

$$D_{\text{res},i} = d_i - p_i, \qquad S_{d,i} = r_i - v_i, \tag{6}$$

$$D_{\text{delivery},i} = f_i - d_i, \qquad S_{\text{action},i} = f_i - r_i, \tag{7}$$

$$L_i^* = f_i - a_i = D_{\text{adm},i} + D_{\text{res},i} + D_{\text{delivery},i}. \tag{8}$$

Only the three D terms in Eq. 8 form an additive partition. Producer lateness, lookup, and enqueue invocation–response service are nested within or overlap admission; dequeue service and the consumer action are nested within the post-publication path and are never added again. Recording linearization boundaries avoids defining residence from operation responses that may overlap across cores. A full row retains a_i, b_i, c_i, u_i, e_i but has no p_i, accepted ordinal, consumer row, or latency. Equation 8 is an identity between non-overlapping timestamp intervals, not a microarchitectural cost partition. The recorded value is

$$L_i^{\mathrm{obs}} = L_i^{*} + \epsilon_i, \tag{9}$$

where ϵ_i includes clock-read, synchronization, quantization, and calibration error. Every timestamp equation is checked exactly in integer ticks after the accepted-ordinal join.

Let a run contain N_r accepted events whose logical arrival a_i lies in the post-warm-up measurement horizon and whose index/payload loads and private checksum update complete either during that horizon or the subsequent drain. Sort their observed latencies as $X_{(1)} \le \cdots \le X_{(N_r)}$, retaining repeated equal values as separate observations. The non-interpolated inverse empirical-CDF estimator is

$$\widehat{Q}_{p,r} = X_{(\lceil pN_r \rceil)}. \tag{10}$$

Thus ties require no special interpolation. Events admitted before the arrival horizon ends remain included when they finish during drain. A dropped event has no latency observation and contributes instead to the separately reported full/drop outcome. Events are never pooled across runs to construct a primary response. The primary run-level response is

$$Y_r = \log \widehat{Q}_{0.999,r}\left(L^{\mathrm{obs}}\right). \tag{11}$$

All reported run quantiles use Eq. 10. Secondary responses are p_{50}, p_{90}, p_{99}, qualifying $p_{99.99}$, maximum with sample count and duration, operation and subpath diagnostics, throughput, occupancy, producer lateness, and mandatory full/drop count and rate. Counter quantities remain diagnostic.

4.3 Treatments and primary estimand

Let the five queue/software packages be $\mathcal{G} = \{R0, R1, R2, L0, L1\}$: ring off, ring one-line, ring latency-matched, linked off, and linked one-hop. Hardware state is $h \in \{H0, H1\}$. Placement P is near or far; working-set residency class W is L2-resident, LLC-resident, or beyond-LLC; and offered load A is 0.25, 0.50, or 0.75 of the frozen service-rate reference. Ring and linked packages use the same logical C and must each satisfy the class criterion, but $F_R(C;P)$ and $F_L(C;P)$ may differ because package-specific metadata and the recycler remain part of the complete-package estimand. No byte-for-byte footprint equality is claimed. Comparisons are valid only within one platform build, immutable payload and event-address schedule, node-order seed where applicable, placement definition, and measurement protocol.

The ring Stage A policy is a bundle: the producer hints the future slot line with write intent where available, and the consumer issues a read-oriented retaining hint for that line. A platform without a documented write-intent hint uses a read-oriented producer fallback and labels the package accordingly; results are not treated as equivalent across those platform forms. The linked $L1$ policy is a consumer-only read-oriented retaining hint targeting the immediate successor node-header line after its address has been acquired. Stage A never prefetches event records or recycler storage. Producer-only and consumer-only ring variants, other locality forms, and event-record prefetch are exploratory ablations.

The primary estimand compares complete queue-design packages. For the linked package this includes recycler removal and return, node initialization, link publication, and unlink. The ring

has no equivalent node recycler. Separate matched diagnostic runs time those linked subpaths and collect aligned counters, but primary runs retain only the end-to-end boundaries. The experiment therefore cannot identify pointer dependence as the sole cause of a queue-family contrast; a passing node-pattern gate and consistent diagnostics provide supporting evidence only.

4.4 Confirmatory hypotheses

Let $\bar{Y}_{g,h,c}$ be the expected run response for package g, hardware state h, and context $c = (P, W, A)$. Define hardware-averaged within-family software effects

$$E_{Rj}(c) = \tfrac{1}{2} \sum_{h} \{\bar{Y}_{Rj,h,c} - \bar{Y}_{R0,h,c}\}, \quad j \in \{1, 2\}, \tag{12}$$

$$E_{L1}(c) = \tfrac{1}{2} \sum_{h} \{\bar{Y}_{L1,h,c} - \bar{Y}_{L0,h,c}\}. \tag{13}$$

The primary representation-specific software contrasts are effect modifications,

$$C_{\mathrm{SW},j} = \langle E_{Rj}(c) - E_{L1}(c) \rangle_c, \tag{14}$$

where $\langle \cdot \rangle_c$ is an equal-weight average over the eighteen Stage A contexts. Equation 14 compares within-family effects, not the absolute means or numeric distances of incomparable policies. Queue-by-hardware, hardware-by-software, and the ring-level contrasts are defined with stable identifiers in Table 8.

The practical bound is denoted δ_* but is not yet frozen. No application latency budget or authorized engineering decision exists in the repository. The candidate $\delta_0 = \log(1.05)$ is therefore a sensitivity value only, not a confirmatory threshold. Before confirmatory execution, a treatment-blind engineering review must select one δ_* using an application latency budget, measured clock capability, blinded baseline variation, instruction and platform-state complexity, and deployment and maintenance cost; treatment labels and effects remain inaccessible. The bound, rationale, and reviewers are recorded, and all H1/H2 and H3 precision calculations and final replication counts are then recomputed. Confirmatory execution and H3 validation cannot begin before this freeze.

H1 (representation-specific prefetch effect modification). H1 contains the seven two-sided contrasts in Table 8: two software effect-modification contrasts, one queue-by-hardware contrast, three hardware-by-software contrasts, and one ring-level contrast. After δ_* is frozen, a contrast is supported only when its 95% max-T simultaneous interval lies wholly above δ_* or below $-\delta_*$. If every interval lies inside $[-\delta_*, \delta_*]$, the family is practically equivalent; other outcomes are inconclusive. A contrast requiring any cell outside the zero-loss regime is non-estimable under the confirmatory estimand. No raw Rj mean is compared with a raw $L1$ mean as though the policies were common treatments.

H2 (context-dependent prefetch response). H2 contains the twenty two-sided contrasts in Table 9. Four defined prefetch effects—package-averaged hardware, E_{R1}, E_{R2}, and E_{L1}—are crossed with far versus near placement, the two adjacent working-set-residency transitions, and the two adjacent load transitions. The two context axes not under test are equal-weight averaged. Support, practical equivalence, regime-gate blocking, and inconclusive outcomes use the H1 rule and the H2-specific max-T family.

H3 (precision-gated held-out selection). H3 is conditionally confirmatory for the six predeclared contexts $\mathcal{C}_{H3} = \{(P, W, A) : A = 0.50\}$ on each platform. For each context, training chooses from the frozen ten candidates in $\mathcal{G} \times \{H0, H1\}$ by minimizing the arithmetic mean of Y_r over `H3_TRAIN` blocks, with the fixed package-first lexicographic tie break defined in Section 6.8. This is equivalent to minimizing the geometric mean of run-level $\widehat{Q}_{0.999,r}$. Before selection, training precision is sized over all 270 context-specific unordered candidate pairs and validation precision over the complete 540-member ordered candidate-minus-alternative family. The selected candidates and their training-record checksum are frozen before the `H3_VALIDATION` namespace is unsealed; validation is never resized after selection. Held-out reporting contains nine paired selected-minus-alternative comparisons per context, or 54 per platform, and requires every one-sided 95% max-T upper limit to be below δ_*. H1/H2 may access all complete role-assigned Stage A blocks only after validation is unsealed, H3 is evaluated, and its access record is sealed. Missing, invalid, zero-loss-regime-failed, or effective-tail-inestimable required cells make confirmatory RQ3 unresolved; no candidate is removed or imputed. Leave-one-training-block-out selection frequency remains supplementary.

MPSC producer count, batching, burstiness, cache or bandwidth interference, alternative data or page placement, memory-order variants, SMT placement, and overload have exploratory hypotheses only. They cannot rescue a failed confirmatory hypothesis or replace a missing Stage A cell.

5 Operational Performance Model

The model organizes possible mechanisms and defines estimable run responses. It does not assert that processor events form a sum of disjoint physical delays.

5.1 Notation

Table 3: Primary notation used in the operational and estimable models.

Symbol	Definition and unit
$a_i, b_i, c_i, u_i, p_i, e_i$	scheduled arrival, handling start, lookup completion, enqueue invocation, publication/linearization, and enqueue response (time)
v_i, d_i, r_i, f_i	successful dequeue invocation, removal/linearization, dequeue response, and consumer-action completion (time)
L_i^*, L_i^{obs}	latent and observed end-to-end latency for event i (time)
ϵ_i	timestamp and calibration error in Eq. 9 (time)
$S_{e,i}, S_{d,i}, S_{\text{lookup},i}, S_{\text{action},i}$	nested enqueue, dequeue, lookup, and consumer-action diagnostics (time)
$O(t)$	accepted, unconsumed events at time t (events)
$\mathcal{G}$	packages $R0, R1, R2, L0, L1$ (ring/linked and nested software policy)
H, P, W, A	hardware state, placement, working-set class, and offered-load level
Y_r	log run-level $p_{99.9}$ end-to-end latency (log time)
δ_*	practical equivalence/non-inferiority bound to be frozen before confirmation (log ratio)
u_m, v_{mb}, g_{mbh}	platform, temporal-block, and hardware whole-plot effects (log time)

5.2 Mechanism map

An event path can expose queue algorithm work, node recycling, immutable event-record access, atomic retries, cache-line ownership transfer, demand-miss latency, remote-memory access, empty/full outcomes, and measurement overhead. Let

$$\mathcal{Z}_i = \{Z_{\text{alg}}, Z_{\text{recycle}}, Z_{\text{event}}, Z_{\text{atomic}}, Z_{\text{coh}}, Z_{\text{miss}}, Z_{\text{remote}}, Z_{\text{state}}, Z_{\text{meas}}\}_i \tag{15}$$

denote observable or latent mechanism markers. The operational relation is

$$L_i^* = G(D_{\mathrm{adm},i}, D_{\mathrm{res},i}, D_{\mathrm{delivery},i}, \mathcal{Z}_i; \mathcal{G}, H, P, W, A)\,. \tag{16}$$

G is not additive: a remote demand miss may also be a stalled interval and a coherence transfer; prefetch can overlap other work; a recycler operation changes both algorithmic work and address traffic. Counters approximate some Z terms at run granularity but cannot assign causal time to an event. Randomized contrasts in Y_r, not counter correlations, carry primary inference.

For the ring, a future slot $k + d$ is available arithmetically, giving the treatment-feasibility condition

$$d\, c_{\mathrm{issue}} \gtrsim \ell_{\mathrm{target}}, \tag{17}$$

where c_{issue} and ℓ_{target} are calibration quantities. Equation 17 selects a level and does not predict improvement. In the linked package, the immediate successor becomes available only after the current link load. Its one-hop hint targets that successor's header line; multi-hop pointers would alter the package. Even with dependent linkage, a regular physical node cycle could assist a stride mechanism. Address-dependence interpretation therefore requires the seed-permutation gate in Section 4; the primary estimand remains the complete queue and reuse package regardless of that gate.

5.3 Full-rank run-level model

The five package levels use four explicit columns rather than an unexplained five-level factor. Let $q = 1$ for linked packages and zero for ring packages, and define x_{R1}, x_{R2}, x_{L1} as indicators for the named positive software policies. Thus the package rows $(q, x_{R1}, x_{R2}, x_{L1})$ are $R0 = (0, 0, 0, 0)$, $R1 = (0, 1, 0, 0)$, $R2 = (0, 0, 1, 0)$, $L0 = (1, 0, 0, 0)$, and $L1 = (1, 0, 0, 1)$. Together with an intercept these rows are full rank. Hardware uses centered coding $h = -1/2$ for $H0$ and $+1/2$ for $H1$; P, W, and A use full-rank sum-to-zero columns so equal-weight marginal effects are explicit.

Let r index one run, m platform, b temporal block, and h_b the hardware whole plot. With $\mathbf{x} = (x_{R1}, x_{R2}, x_{L1})^{\mathsf{T}}$ and context-column vectors $\mathbf{p}, \mathbf{w}, \mathbf{a}$, the confirmatory model is

$$\begin{aligned} Y_{rmbh_b} = {} & \beta_0 + \beta_Q q + \beta_H h + \boldsymbol{\beta}_S^{\mathsf{T}}\mathbf{x} + \boldsymbol{\beta}_P^{\mathsf{T}}\mathbf{p} + \boldsymbol{\beta}_W^{\mathsf{T}}\mathbf{w} + \boldsymbol{\beta}_A^{\mathsf{T}}\mathbf{a} \\ & + q(\boldsymbol{\gamma}_{QP}^{\mathsf{T}}\mathbf{p} + \boldsymbol{\gamma}_{QW}^{\mathsf{T}}\mathbf{w} + \boldsymbol{\gamma}_{QA}^{\mathsf{T}}\mathbf{a}) + \gamma_{HQ} hq + h\boldsymbol{\gamma}_{HS}^{\mathsf{T}}\mathbf{x} \\ & + h(\boldsymbol{\eta}_{HP}^{\mathsf{T}}\mathbf{p} + \boldsymbol{\eta}_{HW}^{\mathsf{T}}\mathbf{w} + \boldsymbol{\eta}_{HA}^{\mathsf{T}}\mathbf{a}) \\ & + \sum_{k \in \{R1, R2, L1\}} x_k(\boldsymbol{\theta}_{kP}^{\mathsf{T}}\mathbf{p} + \boldsymbol{\theta}_{kW}^{\mathsf{T}}\mathbf{w} + \boldsymbol{\theta}_{kA}^{\mathsf{T}}\mathbf{a}) + u_m + v_{mb} + g_{mbh_b} + \varepsilon_{rmbh_b}. \end{aligned} \tag{18}$$

The $h\mathbf{x}$ columns are the required hardware–software interaction $H{:}S(Q)$. No $H{:}S(Q){:}(P, W, A)$ or other unconstrained higher-order interaction is confirmatory; hardware context effects are averaged across packages and software context effects are averaged across hardware states in the H2 definitions. Any condition-specific higher-order interaction is exploratory and reported only with that label.

Table 4: Model terms and their predeclared inferential roles.

Term	Interpretation	RQ/H	Exact contrast identifiers	Status
Q and $\mathbf{x}$	linked-off versus ring-off baseline and three within-family software effects	RQ1/H1	H1-SW-R1, H1-SW-R2, H1-R12	confirmatory
H	changed versus default hardware state for the reference package	RQ1/H1	component of H1-HQ and H1-HS-*	confirmatory
P, W, A	context baselines under sum coding	adjustment	none alone	confirmatory nuisance
Q:(P, W, A)	queue-package baseline effect modification unrelated to prefetch	adjustment	none alone	confirmatory nuisance
H:Q	difference between linked-off and ring-off hardware effects	RQ1/H1	H1-HQ	confirmatory
H:$\mathbf{x}$	hardware modification of each within-family software effect	RQ1/H1	H1-HS-R1, H1-HS-R2, H1-HS-L1	confirmatory
H:(P, W, A)	package-averaged hardware effect across context	RQ2/H2	H2-P-H, H2-W12-H, H2-W23-H, H2-A12-H, H2-A23-H	confirmatory
$\mathbf{x}$:(P, W, A)	hardware-averaged software effects across context	RQ2/H2	all H2-*-R1, H2-*-R2, H2-*-L1	confirmatory
Random effects	platform, temporal block, and hardware whole-plot variation	all	covariance structure only	confirmatory
Higher-order terms	condition-specific HW–SW–context modification	exploratory	no confirmatory identifier	exploratory

u_m is a random platform intercept only with at least three independently characterized platforms; otherwise platform is fixed and inference is platform-conditioned. v_{mb} is a temporal-block intercept and g_{mbh_b} is the whole-plot error, distinct from run error ε. The run is the independent unit. Before analysis, the frozen matrix is checked for exact column rank, empty cells, aliasing with build or seed families, and estimability of every contrast row. A failure blocks the affected confirmatory family; no column or cell is silently removed or imputed.

Residual symmetry and variance homogeneity are checked on the log scale. The predeclared outer cluster bootstrap supplies contrast covariance and simultaneous inference if model-based assumptions fail, while preserving temporal blocks and whole plots. H3 may use the fixed model on training blocks, but its held-out comparison and precision rules are independent of H1/H2 estimation. No fitted coefficient or recommendation is present in this pre-experiment model.

6 Experimental Methodology

6.1 Staged design

Stage A is the mandatory confirmatory core: the bounded SPSC ring and linked 1P1C package, two verified hardware states, five queue-nested software packages, near and far placement, three working-set residency classes, and three sub-saturation loads. It uses one immutable event-record size and observes occupancy and full/drop outcomes. Stage B is an exploratory contention extension using tagged-pointer NBLFQ as MPSC when its representation assumptions hold [3]; it varies producer count and batching and records CAS, scan, fairness, and ownership diagnostics. Stage C contains mutable event preparation and ownership handoff, burstiness, cache or bandwidth interference, alternative data/page placement, memory-order sensitivity, same-core SMT, and overload. Neither extension can substitute for an omitted Stage A cell.

6.2 Queue packages, address order, and correctness contract

Table 5: Mandatory queue packages. The linked timed path includes its regular ring-based recycler.

Property	Bounded SPSC ring	Linked 1P1C package
Primary source	Torquati, Fig. 3 [24]	Torquati dSPSC linkage and node cache, Fig. 6 [24]
Representation	circular event-pointer slots; producer and consumer cursors on separate lines	sentinel-headed linked nodes plus a bounded circular SPSC pointer recycler
Cardinality/capacity	one producer, one consumer; C slots	one producer, one consumer; C events plus one sentinel
Empty/full	distinguished empty slot; try-operation returns false	successor null means empty; unavailable recycled node means full
Allocation/order	fixed ring and common event pool	fixed, line-aligned arena; sentinel and supply order follow a frozen permutation of $C + 1$ nodes
Publication/reuse	release-publish and acquire-observe slot; circular reuse	release-publish and acquire-observe link; old sentinel returns through release/acquire recycler handoff
Timed path	deterministic record-index lookup, slot enqueue/dequeue, index/payload loads, private checksum update	deterministic record-index lookup, recycler obtain/return, node initialization, link/unlink, index/payload loads, private checksum update
Primary interpretation	complete ring package	complete linked-plus-recycler package; pointer-dependence interpretation requires the address-pattern gate

Both packages transfer a pointer to the same one-cache-line event record. A common pool of C records and a frozen event-order permutation are reused across paired package and prefetch runs. Each record contains an immutable index, one fixed aligned 64-bit payload word, and inert padding; the producer never rewrites a record during Stage A. After dequeue, the consumer reads both fields, applies one frozen branch-free fixed-instruction integer-mixing operation to its private 64-bit rolling checksum, and records f_i after the update. The mixing operation is identical for all packages, allocates no memory, and is selected before pilot; generated-code inspection and the post-run checksum establish that the loads and update remain present without volatile, atomic, or mutable record fields. The linked arena is a single virtually contiguous, producer-home allocation of $C + 1$ integral-cache-line nodes. The initial sentinel and recycler order follow the permutation in Section 4; the recycler is FIFO, so zero-loss execution repeats the same cycle. The node arena, event pool, and their mappings persist across paired hardware and software treatments within a temporal block. No record ownership handoff, event-record recycling, mutable payload preparation, allocation, random-number generation, permutation, or sorting occurs during the measured horizon.

The primary memory-order mapping is release publication and acquire observation of queue slots or links, with relaxed operations restricted to thread-owned cursors. Event records require no ownership transfer because they are immutable before the horizon. Consumer release before recycler publication and producer acquire on removal prevent premature linked-node reuse. The later implementation must establish these mappings under its language model and demonstrate atomic lock freedom. Alternative queue orders and mutable-record ownership are Stage C only.

6.3 Frozen Stage A data placement and working sets

Stage A has one data-home policy, applied identically to both packages. Shared queue storage, the linked arena and recycler, and the event-record pool reside on the producer's NUMA node. Producer-

private timing/sample storage is producer-local and consumer-private timing/sample storage is consumer-local. Alternative consumer-local, interleaved, replicated, migrated, or huge-page data policies are Stage C treatments.

Table 6: Mandatory Stage A allocation and first-touch policy.

Structure	Required data home	First-touch responsibility	Verification
Ring slots/control	producer NUMA node	producer initializes every page before warm-up	page-node map before, monitored during, and checked after run
Linked arena/recycler	producer NUMA node	producer initializes nodes and recycler pages in frozen order	same, plus node-map and delta checksums
Event-record pool	producer NUMA node	producer initializes every record page; consumer work does not migrate pages	same map and event-delta checksum for paired packages
Producer-private samples	producer NUMA node	producer	allocation node and page residency
Consumer-private samples	consumer NUMA node	consumer	allocation node and page residency
Arrival schedule/manifest	producer node; immutable	producer for schedule, control thread for manifest	generation, checksum, and I/O outside measurement; deadline reads are common driver work

Near placement uses distinct non-SMT cores in the same NUMA node and the closest LLC-sharing domain. Far placement keeps all shared data producer-local and pins the consumer to a different NUMA node. A platform without both placements is ineligible for confirmatory RQ2; a single-NUMA cross-LLC case may be reported only in Stage C. A documented operating-system mechanism verifies allocation nodes before the run, monitors migration or residency changes during it without touching hot lines, and rechecks them afterward. Any mismatch invalidates the run.

Let $F_q(C;P)$ be the measured shared hot footprint for package q at common logical queue/event-pool capacity C, including slots or nodes, recycler where applicable, event records, padding, and allocation rounding. In general $F_R(C;P) \neq F_L(C;P)$; package-specific metadata and the linked recycler are deliberately retained, and the design makes no byte-for-byte footprint-matching claim. Private sample buffers and immutable manifests are reported separately. Let $K_2(P)$ be the smaller usable L2 capacity of the selected producer and consumer, and let $K_{\text{home}}(P)$ be the usable capacity of the single producer-home LLC domain after documented sharing. For near placement this is the one LLC domain shared by the pair; for far placement it is the producer-node home LLC only. The consumer's remote LLC is never added to it.

For each placement, capacities are resolved independently by the following common-logical-capacity rules; both package footprints must satisfy the same frozen residency-class criterion:

- **L2-resident:** largest power-of-two C for which $4B < F_q(C;P) \leq 0.5K_2(P)$ for both packages;
- **LLC-resident:** largest C for which $2K_2(P) < F_q(C;P) \leq 0.5K_{\text{home}}(P)$ for both packages;
- **beyond LLC:** smallest C for which $F_q(C;P) \geq 2K_{\text{home}}(P)$ for both packages.

Here B is measured line size. A platform is ineligible if a common logical capacity does not exist. These are matched residency classes, not equal byte footprints or claims that every access hits the named cache.

6.4 Prefetch treatments, cells, and feasibility

Table 7 fixes every Stage A hint target. "Retaining" means the platform's documented temporal/local-cache form; exact instruction semantics are recorded rather than assumed portable. Ring $R1$ and $R2$ are bundled producer-plus-consumer policies. Producer-only or consumer-only attribution requires a separately declared exploratory ablation.

Table 7: Stage A software-prefetch sites and target semantics.

Package	Worker	Target	Hint form	Platform-conditioned rule
$R1, R2$	producer	future ring slot cache line	write-intent, retaining	if unavailable, read-oriented retaining fallback is named and analyzed as a distinct platform form
$R1, R2$	consumer	future ring slot cache line	read-oriented, retaining	same resolved distance as producer
$L1$	consumer	immediate successor node-header line	read-oriented, retaining	issued only after acquiring successor address
All	both	event record and payload	no Stage A hint	identical deterministic event stream remains a control
Linked	both	recycler storage	no Stage A hint	recycler effects remain part of the package

For pointer-size slot b_s, ring one-line distance is $d_1 = \lceil B/b_s \rceil$. A treatment-blind ring-off calibration separately measures producer and consumer slot-demand latency and issue interval. Let ℓ^* be the larger of the two frozen conservative latency summaries and c^* the smaller of the two frozen conservative issue-interval summaries. The latency-matched distance, rounded upward to a whole cache-line distance, is

$$d_2 = \min\left(\left\lfloor \frac{C}{4d_1} \right\rfloor d_1, \max\left(2d_1, \left\lceil \frac{\ell^*}{c^* d_1} \right\rceil d_1\right)\right). \tag{19}$$

The calibration is repeated separately for every platform, placement, and working-set capacity for which these quantities differ; the same frozen d_2 is used by producer and consumer in $R2$. If the quarter-cap collapses d_2 to d_1, capacity is revised before freeze or the cell/platform is ineligible. No confirmatory outcome influences distance selection. The linked policy has no numeric distance beyond one acquired successor.

Hardware state $H0$ is the platform default and $H1$ one documented changed state. Read-back and separate regular/pointer behavioral probes must distinguish them; the paper names controlled, unchanged, and unknown engines and never generalizes a partial change to "prefetch off." Hardware state is a whole plot. Its order is counterbalanced across temporal blocks, and the 90 package/placement/working-set/load cells are randomized within each state. Paired seed families fix arrival, node, and event orders.

Service-rate calibration uses the same queue implementation, consumer action, placement, capacity, working-set class, hardware state, and software policy as its Stage A cell, but a continuously ready producer and fixed-duration independent calibration runs are used only to estimate transfer capacity. The response is run-level consumed-event throughput. For every required package/state/context let μ_{cell} be its one-sided 95% lower confidence bound over independent calibration runs, and define $\mu_{\text{ref}} = \min \mu_{\text{cell}}$ over all valid required cells. Candidate open-loop loads are $\{0.25, 0.50, 0.75\}\mu_{\text{ref}}$ and undergo a separate matrix-level zero-loss feasibility probe. Before confirmation, a treatment-blind record freezes the per-cell full-probability estimator, simultaneous confidence level, dependence treatment, minimum acceptable whole-matrix completion probability π_{matrix}, and global load-reduction

rule. If p_c^U is the simultaneous upper bound for one offered event in cell c and $E_c = R_{\text{total}} N_{\text{sched},c}$ its planned exposure, the conservative union-bound lower probability is

$$P_L(\text{no full in Stage A}) = max\left(0, 1 - \sum_c E_c p_c^U\right). \tag{20}$$

This gate accounts for all $180R_{\text{total}}$ runs and scheduled events without assuming independent full outcomes; observing zero pilot drops does not make p_c^U zero. If Eq. 20 is insufficient, all loads are reduced by the frozen rule, capacity is revised, the platform is declared infeasible, or confirmation remains unresolved. Strict confirmatory zero loss is unchanged: no nonzero threshold, cell-specific rate, confirmatory tuning, or repeat after a correctly reconciled full outcome is permitted. Calibration duration, repetitions, rates, capacities, confidence method, and π_{matrix} remain freeze outputs. Five packages, two hardware states, two placements, three working-set residency classes, and three loads give

$$5 \times 2 \times 2 \times 3 \times 3 = 180 \text{ cells per complete block and platform.} \tag{21}$$

After δ_* is frozen, a pilot estimates covariance and max-T critical values separately for the seven H1 contrasts and twenty H2 contrasts. R_{H1} and R_{H2} are the respective smallest complete-block counts of at least 20 whose largest prospective within-family simultaneous half-width is at most $\delta_*/2$, and $R_{12} = \max(R_{H1}, R_{H2})$. There is no combined 27-contrast max-T family. A ceiling of 30 is a Stage A go/no-go boundary for each requirement: if either exceeds it, H1/H2 do not start until the design, bound, or available budget is revised without seeing treatment outcomes. The resulting $180R_{12}$ runs fall within a 3,600–5,400 planning envelope only if the frozen bound and both pilot precision rules pass; that range is not a final count in this pre-freeze draft. H3 has separate uncapped-by-R_{12} training and validation requirements in Section 6.8; its blocks may increase the total.

There is one common pool of complete confirmatory Stage A blocks. Before execution, each block receives one immutable role: `H3_TRAIN`, `H3_VALIDATION`, or `H1H2_SUPPLEMENTAL`. Train, validation, and supplemental blocks occupy role-specific subspaces of one common Stage A block namespace; there is no separate H1–H2 confirmatory namespace. All complete blocks may eventually enter H1/H2, but H1/H2 outcomes remain inaccessible until training has been opened, selected candidates and the selection-record checksum have been frozen, and validation has been unsealed and evaluated. If $R_{12} > R_{\text{train}} + R_{\text{val}}$, the difference is supplemental; otherwise train and validation blocks supply H1/H2 after unsealing. Thus $R_{\text{total}} = \max(R_{12}, R_{\text{train}} + R_{\text{val}})$ and $N_{\text{runs}} = 180R_{\text{total}}$ per platform.

Before Stage A, the pilot freezes the parameterized wall-clock estimate

$$T^{(s)}_{\text{platform}} = N^{(s)}_{\text{runs}} \left(T_{\text{setup}} + T_{\text{warmup}} + T_{\text{measurement}} + T_{\text{drain}} + T_{\text{recovery}}\right) + T^{(s)}_{\text{state changes}} \tag{22}$$

for each study component s: pilot/calibration, H1/H2 primary runs, additional H3 blocks, matched counter runs, Stage B, and Stage C. A documented stand-budget review is a go/no-go gate. Optional counter groups and Stages B/C are narrowed first if the budget is exceeded. Confirmatory cells are never removed after outcomes are observed; an infeasible H3 is narrowed before data or reported exploratory.

6.5 Platform and open-loop run controls

Each platform record includes processor model, stepping, microcode, sockets, physical cores, SMT, frequency/power policy, cache and NUMA topology, line size, memory population, operating system/kernel, compiler/library, full flags, base-page policy, and relevant firmware settings. Affinity,

actual CPU, data home, frequency, temperature, migrations, interrupts, and context switches are verified or observed. Hardware-state readback and behavioral verification repeat at every whole-plot boundary.

Pilot, calibration, confirmatory, diagnostic, and exploratory identifiers occupy disjoint namespaces. Warm-up has its own seed namespace and never enters pilot or confirmatory inference. It uses the same queue/software package, hardware-prefetch state, placement, persistent arena mappings, and consumer action as the following measured run. If $h_{\max}$ is the largest valid pilot tail-indicator correlation horizon in event lags and $\rho_{\min}$ is the slowest valid pilot accepted-event rate in events per unit time, then $\tau_{\text{corr}} = h_{\max}/\rho_{\min}$ has units of time. Warm-up lasts at least $\max(5\text{ s}, 10\tau_{\text{corr}})$.

At the end of warm-up, the driver stops creating warm-up arrivals, drains the queue, and stops both workers at a barrier while retaining every allocated and first-touched arena and mapping. A deterministic logical reset restores ring slots empty with both cursors zero; linked sentinel π_0 with recycler entries $\pi_1, \ldots, \pi_C$ in frozen order; event-order position, run counters, and sample indices zero; occupancy zero; and no accepted warm-up event present. Reset performs no reallocation, remapping, regeneration, or broad payload retouch. Both workers then synchronize at a start barrier and derive measurement t_0 from the frozen clock protocol. Hardware-prefetch and cache history are not explicitly cleared: this is a controlled warm-start with deterministic logical reset. Whole-plot counterbalancing, persistent paired mappings, identical reset logic, and a separately frozen recovery interval control carryover. Measurement address/checksum reports begin at this origin and exclude warm-up activity. The pilot freezes one applicable measurement horizon before confirmation; no run is shortened, extended, or repeated after inspecting treatment-dependent effective count.

The producer waits for each pre-generated deadline in a tight clock-poll loop using the platform's non-sleeping processor-relax hint and never calls sleep, yield, or a scheduler API in Stage A. The consumer repeatedly invokes `try-dequeue`; an empty result executes the same relax hint and immediately retries, without adaptive backoff. One shared driver is specialized before measurement so queue selection introduces neither virtual dispatch nor treatment-dependent hot-path branches; generated machine code is inspected for every package. After attempting all scheduled arrivals, the producer release-publishes an `arrivals_finished` flag on a control cache line separate from queue metadata. The consumer continues until it acquire-observes the flag and the queue is empty, completes drain, and exits. Termination and other control fields never share a cache line with queue hot metadata.

All arrivals, node order, and event-record order are generated before the run, and event records are initialized and checksummed before warm-up. A late producer retains a_i and attempts overdue events in order. Stage A admission includes logical-arrival handling, deterministic pointer/index lookup, and enqueue work; it contains no mutable payload preparation. The manifest reconciles offered, attempted, accepted, full, consumed, and final occupancy counts and records producer accepted/drop indices, consumer-observed record indices, lateness, node/event delta checksums, and maximum occupancy. Exact reconciliation and correctness remain run-validity requirements. A correctly counted full return remains in the run, while $N_{\text{full}} > 0$ marks that required cell outside the strict zero-loss regime and blocks each confirmatory contrast that depends on it.

6.6 Timing and diagnostic counters

The clock must be monotonic across selected cores, have stable conversion, and pass skew, drift, resolution, serialization, and read-cost gates. Timestamp boundaries prevent compiler movement without strengthening queue order. Boundary overhead is calibrated as a distribution; corrected and uncorrected values are retained. Every pilot, calibration, and confirmatory Stage A latency run writes ordered per-event observations to preallocated producer-private and consumer-private buffers.

Every row carries run identity. Producer rows contain logical sequence, record index, a_i, b_i, c_i, u_i, e_i, attempted and accepted/full status, and, only when accepted, p_i and accepted ordinal. Consumer rows contain consumed ordinal, observed record index, v_i, d_i, r_i, f_i. After the run, FIFO order defines accepted ordinal, producer and consumer row k must match, timestamp ordering and every equation in Section 4 must hold, and only then is latency computed. A join audit is retained whether it passes or fails; successful joined rows additionally retain source-row ordinals and all derived intervals. A full row retains its attempt boundaries but has no accepted ordinal, consumer row, or latency.

The row schema is logical rather than physical. A small envelope records logical-schema and physical-format IDs, encoding, integer time unit, endianness, compression, row and byte counts, immutable order, SHA-256, an external artifact reference, and the phase/integrity reference; inline JSON rows are test-only. Let N_s and N_a be scheduled and accepted rows and b_P, b_C, b_J the later-frozen physical row sizes. The measured buffer satisfies $B_{\text{hot}} \geq N_s b_P + N_a b_C$, conservatively $N_s(b_P + b_C)$, while $N_a \geq N_{\text{eff}} \geq 200{,}000$ is necessary for an estimable primary tail. Across $180R_{\text{total}}$ runs, durable and temporary storage is budgeted symbolically from these row counts, physical sizes, joined rows, and frozen copy multipliers. Compression occurs only after measurement and receives no credit in the hot-buffer bound. Raw rows become immutable after sealing. Histograms, CCDFs, and quantiles are derived artifacts and never replace the primary ordered data. Counter-only diagnostic runs may use smaller products only when the corresponding primary latency run exists and the diagnostic contract requires no ordered event analysis. No percentile, allocation, formatted output, or file I/O occurs on the hot path.

Primary latency runs collect only validity metadata. Matched diagnostic runs use the same cell and seeds but may add timing boundaries for deterministic immutable-record pointer lookup and, for the linked package, recycler obtain, node initialization, link publication, unlink/head advance, and recycler return. There is no Stage A event-record obtain/return boundary. Those diagnostics are never enabled in a primary run. Counter groups cover demand fills, prefetch requests/use where exposed, stalls and memory-level parallelism, coherence transfer, local/remote traffic, and disturbances; Stage B adds CAS and scan diagnostics. Queue metadata, linked recycler, and event-record address ranges are separated in address filters or sampling attribution where the platform permits. Each event's encoding, domain, multiplexing, skid, and limitation are recorded. Diagnostic associations cannot replace randomized end-to-end contrasts.

6.7 Statistical protocol and exact contrasts

One complete run is the independent unit. Event observations estimate its quantile by Eq. 10 but are not independent replications and are never pooled across runs. All primary families use 95% coverage after δ_* is frozen. Differential tail amplification relative to the median is

$$\Delta_{\text{tail}} = \log \frac{\widehat{Q}_{0.999,t}}{\widehat{Q}_{0.999,0}} - \log \frac{\widehat{Q}_{0.50,t}}{\widehat{Q}_{0.50,0}}, \tag{23}$$

computed from run summaries rather than pooled events, where t denotes the named treatment and 0 its predeclared within-contrast baseline.

For compact definitions let $\Delta_H(g,c) = \bar{Y}_{g,H1,c} - \bar{Y}_{g,H0,c}$ and $E_H(c) = |\mathcal{G}|^{-1} \sum_{g \in \mathcal{G}} \Delta_H(g,c)$. E_{R1}, E_{R2}, and E_{L1} are defined in Section 4.4. Angle brackets denote equal weighting over the named nuisance axes. Every row of Tables 8 and 9 is two-sided, uses bounds $[-\delta_*, \delta_*]$, and belongs respectively to the H1 or H2 max-T multiplicity family; no direction is predicted.

Table 8: The seven exact H1 contrasts. $\langle\cdot\rangle_c$ averages all placement, working-set, and load cells.

Identifier	Mathematical definition	Fixed/averaged factors	Interpretation
H1-SW-R1	$\langle E_{R1}(c) - E_{L1}(c)\rangle_c$	SW within family; average H, P, W, A	one-line ring versus linked one-hop effect modification
H1-SW-R2	$\langle E_{R2}(c) - E_{L1}(c)\rangle_c$	SW within family; average H, P, W, A	latency-matched ring versus linked one-hop effect modification
H1-HQ	$\langle \Delta_H(L0, c) - \Delta_H(R0, c)\rangle_c$	SW off; average P, W, A	queue modification of hardware effect
H1-HS-R1	$\langle \Delta_H(R1, c) - \Delta_H(R0, c)\rangle_c$	ring; average P, W, A	hardware modification of ring one-line effect
H1-HS-R2	$\langle \Delta_H(R2, c) - \Delta_H(R0, c)\rangle_c$	ring; average P, W, A	hardware modification of ring latency-matched effect
H1-HS-L1	$\langle \Delta_H(L1, c) - \Delta_H(L0, c)\rangle_c$	linked; average P, W, A	hardware modification of linked one-hop effect
H1-R12	$\langle E_{R2}(c) - E_{R1}(c)\rangle_c$	ring; average H, P, W, A	difference between positive ring lookaheads

Define five context operators: $D_P(e) = \langle e(\text{far}, W, A) - e(\text{near}, W, A)\rangle_{W,A}$; $D_{W12}(e) = \langle e(P, \text{LLC}, A) - e(P, \text{L2}, A)\rangle_{P,A}$; $D_{W23}(e) = \langle e(P, \text{beyond}, A) - e(P, \text{LLC}, A)\rangle_{P,A}$; $D_{A12}(e) = \langle e(P, W, 0.50) - e(P, W, 0.25)\rangle_{P,W}$; and $D_{A23}(e) = \langle e(P, W, 0.75) - e(P, W, 0.50)\rangle_{P,W}$.

Table 9: The twenty exact H2 contrasts. Hardware effects average the five packages; software effects average $H0/H1$. The two context axes not named by an operator are equal-weight averaged.

Identifier	Definition	Package/treatment	Context interpretation
H2-P-H	$D_P(E_H)$	changed–default, average $\mathcal{G}$	hardware effect, far versus near
H2-P-R1	$D_P(E_{R1})$	ring one-line–off, average H	software effect, far versus near
H2-P-R2	$D_P(E_{R2})$	ring matched–off, average H	software effect, far versus near
H2-P-L1	$D_P(E_{L1})$	linked one-hop–off, average H	software effect, far versus near
H2-W12-H	$D_{W12}(E_H)$	changed–default, average $\mathcal{G}$	hardware effect, LLC versus L2
H2-W12-R1	$D_{W12}(E_{R1})$	ring one-line–off, average H	software effect, LLC versus L2
H2-W12-R2	$D_{W12}(E_{R2})$	ring matched–off, average H	software effect, LLC versus L2
H2-W12-L1	$D_{W12}(E_{L1})$	linked one-hop–off, average H	software effect, LLC versus L2
H2-W23-H	$D_{W23}(E_H)$	changed–default, average $\mathcal{G}$	hardware effect, beyond versus LLC
H2-W23-R1	$D_{W23}(E_{R1})$	ring one-line–off, average H	software effect, beyond versus LLC
H2-W23-R2	$D_{W23}(E_{R2})$	ring matched–off, average H	software effect, beyond versus LLC

Identifier	Definition	Package/treatment	Context interpretation
H2-W23-L1	$D_{W23}(E_{L1})$	linked one-hop–off, average H	software effect, beyond versus LLC
H2-A12-H	$D_{A12}(E_H)$	changed–default, average $\mathcal{G}$	hardware effect, 0.50 versus 0.25 load
H2-A12-R1	$D_{A12}(E_{R1})$	ring one-line–off, average H	software effect, 0.50 versus 0.25 load
H2-A12-R2	$D_{A12}(E_{R2})$	ring matched–off, average H	software effect, 0.50 versus 0.25 load
H2-A12-L1	$D_{A12}(E_{L1})$	linked one-hop–off, average H	software effect, 0.50 versus 0.25 load
H2-A23-H	$D_{A23}(E_H)$	changed–default, average $\mathcal{G}$	hardware effect, 0.75 versus 0.50 load
H2-A23-R1	$D_{A23}(E_{R1})$	ring one-line–off, average H	software effect, 0.75 versus 0.50 load
H2-A23-R2	$D_{A23}(E_{R2})$	ring matched–off, average H	software effect, 0.75 versus 0.50 load
H2-A23-L1	$D_{A23}(E_{L1})$	linked one-hop–off, average H	software effect, 0.75 versus 0.50 load

H1 and H2 use separate standardized complete-block bootstrap max-T procedures. The bootstrap replicate count B_{boot} and random seed are frozen before confirmatory analysis. For either frozen family, let $\widehat{\mathbf{C}}$ be the original fitted contrast vector. The procedure is:

1. For replicate $b = 1, \ldots, B_{\text{boot}}$, resample complete temporal blocks with replacement within platform, preserving both hardware whole plots, every cell, the paired arrival/node/event seeds, and the split-plot structure.
2. Refit the frozen model to that resample and store the complete vector $\widehat{\mathbf{C}}^{(b)}$; no cell, coefficient, or contrast is selected within a replicate.
3. Estimate $\widehat{\Sigma}_C$ as the sample covariance of the B_{boot} contrast vectors and set $\widehat{s}_j = \sqrt{(\widehat{\Sigma}_C)_{jj}}$.
4. Form $T^{(b)} = \max_j |\widehat{C}_j^{(b)} - \widehat{C}_j|/\widehat{s}_j$ and let $c_{0.95}$ be its empirical 95th percentile.
5. Report $\widehat{C}_j \pm c_{0.95}\widehat{s}_j$ for every member of the family. Practical equivalence requires the whole adjusted interval inside $[-\delta_*, \delta_*]$.

This standardization is the only variance layer in primary H1/H2 intervals. Ordinary intervals and Holm-adjusted p-values are not described as simultaneous.

The event-level moving-block analysis is not nested inside primary contrast bootstraps. The pilot uses it only to estimate within-run effective tail count, choose the measurement horizon, and check stability of Eq. 10; half/double block-length results are sensitivity analyses. The event block length is twice the largest tail-indicator correlation horizon, where the horizon is the first lag after which absolute autocorrelation stays below $1.96/\sqrt{N}$ for ten lags. Primary $p_{99.9}$ requires $N_{\text{eff}} \geq 200{,}000$; $p_{99.99}$ requires $N_{\text{eff}} \geq 2{,}000{,}000$. The horizon is frozen from treatment-blind pilot data and applied unchanged to the applicable confirmatory runs. A genuine, correctly recorded $N_{\text{eff}} < 200{,}000$ is retained, not extended or repeated, and fails the effective-tail estimability gate; its cell and every dependent confirmatory contrast are non-estimable. Insufficient effective count for $p_{99.99}$ suppresses only that secondary estimate. Only actual sample loss, buffer overflow, process interruption, corrupt output, or another correctness/measurement failure may invalidate a run. A correctly reconciled full return, genuine low effective count, and an extreme valid latency are never exclusion reasons. Sensitivity analyses restore disturbance-only exclusions, vary the event block length by one half and two, examine run-quantile stability, and report uncorrected timestamps.

6.8 H3 training and validation gate

H3 uses the preassigned `H3_TRAIN` and `H3_VALIDATION` role subspaces of the common Stage A block namespace and the frozen candidate order

$$\begin{aligned}&(R0,H0),(R0,H1),(R1,H0),(R1,H1),(R2,H0),\\&(R2,H1),(L0,H0),(L0,H1),(L1,H0),(L1,H1).\end{aligned} \tag{24}$$

This is package-first lexicographic order using the declared $\mathcal{G}$ order and $H0 < H1$. Selection is performed independently for each of the six $A = 0.50$ contexts. For candidate k and context c, training computes

$$\widehat{\mu}_{k,c}^{\text{train}} = \frac{1}{R_{\text{train}}} \sum_{b \in \mathcal{B}_{\text{train}}} Y_{kcb}. \tag{25}$$

It selects the smallest $\widehat{\mu}_{k,c}^{\text{train}}$ and uses Eq. 24 for an exact numerical tie. Because $Y = \log \widehat{Q}_{0.999}$, this minimizes the geometric mean of the run-level $p_{99.9}$ estimates. Validation data, adaptive features, counters, and treatment-dependent removal are not used. A missing, invalid, zero-loss-regime-failed, or effective-tail-inestimable required training cell makes confirmatory H3 unresolved; no value is imputed and no candidate is removed.

R_{train} is at least 12 and is sized before selection from all $\binom{10}{2} = 45$ unordered candidate differences in each of six contexts, or 270 prospective differences. It is the smallest full-rank count whose largest prospective standard error over that complete set is no larger than $\delta_*/2$. R_{val} is at least 8 and is sized before selection from all $6 \times 10 \times 9 = 540$ ordered candidate-minus-alternative comparisons. It is the smallest count whose largest one-sided max-T simultaneous half-width over that complete family is no larger than $\delta_*/2$. Both calculations use blinded pilot covariance, the frozen candidate set, and the frozen δ_* without training-effect or validation-outcome access. Validation is not resized after selection; only the resulting 54 selected-minus-alternative comparisons are reported. The sum of the counts is not capped at 30 merely because H1/H2 are. These counts cannot be finalized and validation cannot begin while δ_* is unresolved.

Data access is chronological and auditable. First, all block roles, seeds, candidate rules, B_{boot}, bootstrap seed, δ_*, and the 270/540 sizing records are frozen while validation artifacts remain sealed. Second, only training artifacts are opened and one candidate for each of the six stable context keys, all training input IDs/hashes, the rule version, and the selection checksum are written to an immutable selection record. Third, an authorized validation-custodian record links the selection ID/hash and validation namespace/artifact ID/hash, names at least one affected validation block, and transitions from `SELECTION_FROZEN` to `VALIDATION_UNSEALED`. For each context and validation block, the selected candidate is paired with each of the other nine candidates, giving differences $D_{jcb} = Y_{\text{selected},cb} - Y_{jcb}$. The validation bootstrap resamples complete validation blocks and applies the analogue of the H1/H2 procedure with $T_+^{(b)} = \max_{j,c}(\widehat{D}_{jc}^{(b)} - \widehat{D}_{jc})/\widehat{s}_{jc}$. Each upper limit is $\widehat{D}_{jc} + c_{0.95,+}\widehat{s}_{jc}$, and all must be below δ_*. A missing, invalid, zero-loss-regime-failed, or effective-tail-inestimable required validation cell makes H3 unresolved. H3 evaluation and H1/H2 release use distinct source-hashed transition records. Only after validation is evaluated and the access record is sealed may H1/H2 use every complete train, validation, and supplemental Stage A block. H1/H2 analysis before that point is a leakage failure. If precision or stand budget fails, H3 is not tested confirmatorily; any narrowing occurs by a treatment-blind amendment before outcomes are opened. Leave-one-training-block-out frequency is supplementary and cannot replace held-out precision.

The pre-freeze total remains symbolic: $R_{\text{total}} = \max(R_{12}, R_{\text{train}} + R_{\text{val}})$ and $N_{\text{runs}} = 180R_{\text{total}}$ per platform. None of these repetition counts is final before its precision and budget gates pass.

6.9 Correctness and validity gates

Both objects must pass FIFO semantics, long concurrent sequence, empty/full boundary, wraparound/reuse, data-race and undefined-behavior, and delayed-worker progress tests. The fixed-arena linked package additionally requires a refinement argument for full semantics; exclusive node-state ownership; no recycle while reachable; exact reproduction of the $C + 1$ permutation; the successor-delta, distinct-line/page, unit/modal-stride, period, and paired-checksum gates; and repeated recycler stress. Event-record gates require pre/post-horizon equality of every record checksum; absence of writes to record payload, timestamp, sequence, or ownership fields; exact frozen record-index order; presence of the expected index/payload loads and rolling-checksum update in generated code; and equality between producer-accepted and consumer-observed streams after joining by accepted ordinal. A count mismatch, unexpected pointer, index mismatch, duplicate, omission, forbidden reordering, or mutated record fails correctness. Reset/start phase checks must also establish the frozen logical origin and absence of warm-up events before measurement.

Every run gates worker affinity, producer-home shared-data placement, consumer-local private buffers, hardware state, clock stability, address-sequence identity, phase/reset identity, count reconciliation, ordered-buffer integrity, and environment. Its manifest distinguishes planned, pre-run, warm-up, reset, measurement-started, measurement-failure, drain-failure, and completed states, plus not-attempted, failed, or passed join. An early failure requires a failure record but no fabricated raw artifact; a failed join retains its audit but no successful joined dataset. A valid completed Stage A run requires both raw streams, passed join evidence, joined rows, complete counts, provenance, and a phase/integrity artifact containing the final rolling checksum, pre/post record-content checksums, ordered-index checksum, address-delta checksum, and their algorithm/version identifiers.

Each failed run keeps its identifier and reason. A correctness or measurement failure makes its original 180-cell temporal block incomplete for primary inference; no cell is replaced inside that block. The complete original block record and all of its valid and invalid runs are retained. If the pre-frozen $R_{\text{replacement,max}}$ budget and replacement authority permit, a new complete 180-cell block receives a new block identifier and ordinal, the same immutable role, a new role-compatible seed subspace, and newly randomized whole-plot/cell order. It never reuses the failed identity. Original plans store explicit null replacement fields; replacement plans store both nonempty linkage/authorization IDs. A semantic validator, rather than array uniqueness alone, proves the exact 180-cell Cartesian product and replacement lineage. An incomplete original block is diagnostic or sensitivity-only and never enters the primary complete-block bootstrap. Exceeding the replacement budget stops confirmatory collection and leaves the study unresolved. Table 10 separates invalidity from observed estimability/regime failures.

6.10 Planned presentation and threats to validity

The later results phase will contain latency CCDFs and percentile–load curves grouped by package and hardware state; mandatory full/drop counts and rates beside every latency panel; forest plots keyed to all estimable H1/H2 identifiers and explicit blocked-regime markers; ring $R1/R2$ and linked $L1$ within-family effects; node- and event-delta diagnostics; placement and working-set-residency interactions; occupancy and lateness; separated queue/recycler/event counter panels; linked diagnostic subpath intervals; and H3 held-out upper intervals for the six declared contexts. A decision table is permitted only if the H3 precision, zero-loss, and non-inferiority gates pass and is limited to those contexts.

Internal validity is threatened by incomplete hardware-state control, clock error, data migration, address-pattern drift, warm-start carryover, instrumentation, validation leakage, and

Table 10: Classification of invalidity and observed estimability/regime outcomes.

Category	Examples	Consequence
Correctness or measurement failure	duplicate or reordered event, invalid clock, wrong placement, sample loss or buffer overflow	run invalid; original block incomplete; only a new complete block may be scheduled within the frozen budget
Count reconciliation failure	any offered, attempted, accepted, full, consumed, or occupancy identity mismatch	protocol failure; run invalid and never repaired by deleting observations
Observed queue-full outcome	correctly counted $N_{\text{full}} > 0$ with identities intact	run retained and reported with accepted-event latencies and mandatory full/drop count and rate; no automatic repeat
Zero-loss regime failure	any required confirmatory cell has a retained full outcome	cell marked outside the estimand; dependent H1/H2 contrasts are non-estimable and H3 is unresolved
Effective-tail insufficiency	genuine $N_{\text{eff}} < 200{,}000$ with complete ordered data	run retained and not extended or repeated; p99.9 cell and dependent contrasts are non-estimable

outcome-dependent selection. Readback, persistent paired mappings, deterministic logical reset, recovery/counterbalancing, pattern checks, sealed access records, diagnostic-only instrumentation, and retaining full and genuine low-N_{eff} outcomes address these risks. Construct validity is limited because the primary queue contrast bundles unequal package footprints, metadata, and reuse work: the packages share logical capacity and residency class, not byte count; the linked recycler is regular and the package is not a pure linked-only path. Address diagnostics can support but not isolate pointer dependence. Immutable records and the fixed rolling-checksum action remove Stage A ownership ambiguity but exclude mutable preparation costs. External validity is platform-conditioned and excludes hard real-time guarantees, other data-home policies, allocators, mutable payload lifecycles, and queue algorithms. Conclusion validity depends on the frozen practical bound, zero-loss and effective-tail estimability gates, complete-block replacement budget, full-rank design, max-T coverage, independent sealed H3 validation, and the wall-clock feasibility gate.

7 Conclusion

This pre-freeze protocol specifies a controlled comparison of complete ring and linked-plus-recycler queue packages under verified prefetch states, producer-home placement, common logical capacity, matched working-set residency class, immutable event-record access, and open-loop offered load. Its methodological contribution is the combination of explicit queue-package estimands, a deterministic warm-start reset, ordered invocation/linearization/response observations joined by accepted ordinal, a non-interpolated run-level tail response, outcome-retaining full/drop and effective-count semantics, a prospective whole-matrix zero-loss feasibility bound, separate H1/H2 complete-block max-T families, and sealed held-out selection whose 270/540 precision sets are fixed before candidate choice. The design does not treat pointer dependence as the sole causal difference between packages, and it claims no empirical winner, effect size, or recommendation.

Before pilot and confirmatory execution, the project must freeze the physical raw encoding and deterministic primitives, platform and hardware-state controls, and every implementation/correctness gate; then δ_*, final capacities and offered loads, matrix-level zero-loss inputs, family-specific replication and replacement-block limits, bootstrap count and seed, measurement/recovery horizons, role-specific seed namespaces, sealing authority, and stand budget. Author and submission metadata also remain unresolved. Results and Discussion will be added only after valid measurements and the predeclared access, regime, correctness, precision, and reproducibility gates have been applied;

until then, the present conclusion is methodological rather than empirical.

References

[1] Irina Calciu, Siddhartha Sen, Mahesh Balakrishnan, and Marcos K. Aguilera. Black-box concurrent data structures for NUMA architectures. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 207–221. Association for Computing Machinery, 2017.

[2] Tudor David, Rachid Guerraoui, and Vasileios Trigonakis. Everything you always wanted to know about synchronization but were afraid to ask. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles*, pages 33–48. Association for Computing Machinery, 2013.

[3] Alexandre Denis and Charles Goedefroit. NBLFQ: A lock-free MPMC queue optimized for low contention. In *2025 IEEE International Parallel and Distributed Processing Symposium*, pages 962–973. IEEE, 2025.

[4] Eiman Ebrahimi, Onur Mutlu, Chang Joo Lee, and Yale N. Patt. Coordinated control of multiple prefetchers in multi-core systems. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 316–326. Association for Computing Machinery, 2009.

[5] Steffen Friedrich, Wolfram Wingerath, and Norbert Ritter. Coordinated omission in NoSQL database benchmarking. In *BTW 2017 Workshopband*, volume P-266 of *Lecture Notes in Informatics*, pages 215–225. Gesellschaft fuer Informatik, 2017.

[6] John Giacomoni, Tipp Moseley, and Manish Vachharajani. FastForward for efficient pipeline parallelism: A cache-optimized concurrent lock-free queue. In *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 43–52. Association for Computing Machinery, 2008.

[7] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 ACM SIGPLAN International Symposium on Memory Management*, pages 63–74. Association for Computing Machinery, 2013.

[8] Giorgos Kappes and Stergios V. Anastasiadis. A family of relaxed concurrent queues for low-latency operations and item transfers. *ACM Transactions on Parallel Computing*, 9(4):16:1–16:37, 2022.

[9] Jaekyu Lee, Hyesoon Kim, and Richard W. Vuduc. When prefetching works, when it doesn't, and why. *ACM Transactions on Architecture and Code Optimization*, 9(1):2:1–2:29, 2012.

[10] Jialin Li, Naveen Kr. Sharma, Dan R. K. Ports, and Steven D. Gribble. Tales of the tail: Hardware, OS, and application-level sources of tail latency. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 9:1–9:14. Association for Computing Machinery, 2014.

[11] Chi-Keung Luk and Todd C. Mowry. Compiler-based prefetching for recursive data structures. In *Proceedings of the 7th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 222–233. Association for Computing Machinery, 1996.

[12] Fabian Mahling, Marcel Weisgut, and Tilmann Rabl. Fetch me if you can: Evaluating CPU cache prefetching and its reliability on high-latency memory. In *Proceedings of the 21st International Workshop on Data Management on New Hardware*, pages 1–9. Association for Computing Machinery, 2025.

[13] Maged M. Michael. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Transactions on Parallel and Distributed Systems*, 15(6):491–504, 2004.

[14] Maged M. Michael and Michael L. Scott. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the 15th Annual ACM Symposium on Principles of Distributed Computing*, pages 267–275. Association for Computing Machinery, 1996.

[15] Daniel Molka, Daniel Hackenberg, Robert Schöne, and Matthias S. Müller. Memory performance and cache coherency effects on an intel Nehalem multiprocessor system. In *Proceedings of the 18th International Conference on Parallel Architectures and Compilation Techniques*, pages 261–270. IEEE, 2009.

[16] Adam Morrison and Yehuda Afek. Fast concurrent queues for x86 processors. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 103–112. Association for Computing Machinery, 2013.

[17] Ruslan Nikolaev. A scalable, portable, and memory-efficient lock-free FIFO queue. In *33rd International Symposium on Distributed Computing (DISC 2019)*, volume 146 of *Leibniz International Proceedings in Informatics*, pages 28:1–28:16. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019.

[18] Ruslan Nikolaev and Binoy Ravindran. wCQ: A fast wait-free queue with bounded memory usage. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 307–319. Association for Computing Machinery, 2022.

[19] Thomas Preud'homme, Julien Sopena, Gaël Thomas, and Bertil Folliot. BatchQueue: Fast and memory-thrifty core to core communication. In *Proceedings of the 22nd International Symposium on Computer Architecture and High Performance Computing*, pages 215–222. IEEE, 2010.

[20] Amir Roth, Andreas Moshovos, and Gurindar S. Sohi. Dependence-based prefetching for linked data structures. In *Proceedings of the 8th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 115–126. Association for Computing Machinery, 1998.

[21] Amir Roth and Gurindar S. Sohi. Effective jump-pointer prefetching for linked data structures. In *Proceedings of the 26th Annual International Symposium on Computer Architecture*, pages 111–121. IEEE Computer Society, 1999.

[22] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation*, pages 239–252. USENIX Association, 2006.

[23] Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N. Patt. Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers. In *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, pages 63–74. IEEE, 2007.

[24] Massimo Torquati. Single-producer/single-consumer queues on shared cache multi-core systems. Technical Report TR-10-20, University of Pisa, Department of Computer Science, 2010.

[25] Junchang Wang, Kai Zhang, Xinan Tang, and Bei Hua. B-Queue: Efficient and practical queuing for fast core-to-core communication. *International Journal of Parallel Programming*, 41(2):137–159, 2013.

[26] Feng Xue, Junliang Wu, Chenji Han, Xinyu Li, Tingting Zhang, Tianyi Liu, and Fuxin Zhang. Augur: Semantics-aware temporal prefetching for linked data structure. *ACM Transactions on Architecture and Code Optimization*, 22(3):117:1–117:27, 2025.